Drupal, as a large and modular CMS/framework, follows **multiple design patterns**—not just one. While there's no official number listed in the documentation, Drupal incorporates **many well-known object-oriented design patterns**, especially from the **Gang of Four (GoF)** book and modern **Symfony/PSR standards**, since Drupal 8+ is built heavily on Symfony components.

🧩 Common Design Patterns Used in Drupal

Here's a list of major design patterns that **Drupal (especially from version 8 onwards)** uses, along with brief explanations and examples:

✅ 1. Singleton Pattern

- **Used for**: Ensuring only one instance of a class exists (e.g., service containers).
- **Example**: The Drupal service container (`\Drupal::service()`) is a singleton.

✅ 2. Factory Pattern

- **Used for**: Object creation logic encapsulation.
- **Example**: `EntityTypeManager::getStorage()` acts as a factory to return storage handlers for entities.

✅ 3. Dependency Injection

- **Used for**: Managing dependencies cleanly.
- **Example**: Services injected via constructors in controllers or plugins.

✅ 4. Strategy Pattern

- **Used for**: Changing behavior at runtime.
- **Example**: Cache backends (`database`, `redis`, `memcache`) are interchangeable strategies implementing a common interface.

✅ 5. Observer Pattern

- **Used for**: Event-driven systems.
- **Example**: Event subscribers/listeners in the Symfony EventDispatcher system.

✅ 6. Plugin Pattern *(Drupal-specific extension of Strategy/Factory)*

- **Used for**: Extensible behavior (e.g., blocks, field types, image effects).
- **Example**: Every block is a plugin; plugins are discovered via annotations or YAML.

✅ 7. Decorator Pattern

- **Used for**: Extending behavior without modifying the original class.
- **Example**: Services can be decorated via the `decorates:` keyword in service definitions.

✅ 8. Adapter Pattern

- **Used for**: Wrapping different interfaces in a unified way.
- **Example**: Drupal adapters for external libraries (like Guzzle HTTP client).

✅ 9. Template Method Pattern

- **Used for**: Base classes providing a workflow with customizable steps.
- **Example**: Form API's `FormBase` class defines the form submission flow.

✅ 10. Chain of Responsibility

- **Used for**: Passing a request along a chain until handled.
- **Example**: Middleware in the HTTP kernel pipeline (e.g., page cache, authentication, etc.).

✅ 11. Command Pattern

- **Used for**: Encapsulating actions as objects.
- **Example**: Symfony Console commands used in Drush or Drupal Console.

✅ 12. Proxy Pattern

- **Used for**: Lazy loading services.
- **Example**: Lazy services defined in the service container (`lazy: true`).

✅ 13. MVC Pattern (Model-View-Controller)

- **Used for**: Separating application concerns.
- **Example**: Controllers (routes), Twig templates (views), and entities/config (model).

✅ 14. State Pattern

- **Used for**: Managing state-based behavior.
- **Example**: Workflow states or user role permissions.

📝 Summary Table

Pattern Name	Where Used in Drupal
Singleton	Service container
Factory	Entity storage, services
Dependency Injection	Controllers, services
Strategy	Cache backends, logging
Observer	Event subscribers
Plugin	Blocks, Fields, Text Filters
Decorator	Service decoration
Adapter	3rd-party library integration
Template Method	FormBase, ControllerBase
Chain of Responsibility	HTTP middleware
Command	Console commands
Proxy	Lazy services
MVC	Routing system
State	Workflow, content moderation

🧠 Conclusion

Drupal uses **at least 14–15 major design patterns**, many of which are **core to its architecture** and essential for writing custom modules, themes, and services.

If you're developing in Drupal (especially 8, 9, or 10), understanding these patterns is **key to writing clean, reusable, and testable code**.

Let me know if you'd like a **code example** of any pattern in Drupal or a guide to how to implement your own **plugin or service**.

The **hierarchy (or tree structure)** of PHP design patterns typically follows the same structure as in other object-oriented programming (OOP) languages, especially based on the **Gang of Four (GoF)** classification. These patterns are broadly categorized into **three main types**, which can be visualized as a **tree or layered structure**.

🌳 Hierarchy of PHP Design Patterns

Here's a tree-structured view of the major **design pattern categories** with specific examples relevant to PHP:

```
Design Patterns
│
├── 1. Creational Patterns (How objects are created)
│   ├── Singleton
│   ├── Factory Method
│   ├── Abstract Factory
│   ├── Builder
│   └── Prototype
│
├── 2. Structural Patterns (How objects are composed)
│   ├── Adapter
│   ├── Bridge
│   ├── Composite
│   ├── Decorator
│   ├── Facade
│   ├── Flyweight
│   └── Proxy
│
└── 3. Behavioral Patterns (How objects interact)
    ├── Chain of Responsibility
    ├── Command
    ├── Interpreter
    ├── Iterator
    ├── Mediator
    ├── Memento
    ├── Observer
    ├── State
    ├── Strategy
    ├── Template Method
    └── Visitor
```

📌 PHP-Specific Usage Highlights

Category	Common Patterns in PHP	Real-World PHP Examples
Creational	Singleton, Factory, Builder	Service containers, Object factories
Structural	Adapter, Decorator, Facade	Symfony Console, Middleware, Drupal plugins
Behavioral	Observer, Strategy, Command	Event Dispatcher, Logger, Symfony Commands

---

🔍 Quick Explanation of Each Type

🧱 1. Creational Patterns

Handle object creation logic.

- **Singleton**: One global instance (e.g., database connection).
- **Factory Method**: Creates objects without exposing the instantiation logic.
- **Abstract Factory**: Creates families of related objects.
- **Builder**: Constructs complex objects step-by-step.
- **Prototype**: Clones existing objects.

🧩 2. Structural Patterns

Handle class and object composition.

- **Adapter**: Makes incompatible interfaces work together.
- **Bridge**: Separates abstraction from implementation.
- **Composite**: Treats individual objects and composites uniformly.
- **Decorator**: Adds behavior dynamically.
- **Facade**: Simplifies complex subsystems.
- **Flyweight**: Reduces object creation for similar objects.
- **Proxy**: Controls access to another object.

🔄 3. Behavioral Patterns

Handle communication and responsibility between objects.

- **Chain of Responsibility**: Pass request along chain of handlers.
- **Command**: Encapsulates a request as an object.
- **Interpreter**: Implements a grammar or language interpreter.
- **Iterator**: Sequentially accesses elements of a collection.
- **Mediator**: Reduces coupling between components.
- **Memento**: Captures and restores object state.
- **Observer**: Notifies dependents of state changes.
- **State**: Changes behavior based on internal state.
- **Strategy**: Chooses an algorithm at runtime.
- **Template Method**: Defines a skeleton algorithm.
- **Visitor**: Separates operations from object structure.

📁 Total: 23 Core Design Patterns

These are the standard GoF patterns, and all of them are relevant and usable in PHP. Frameworks like **Laravel**, **Symfony**, and **Drupal** apply many of them internally.

In **PHP** and **Drupal**, several **design patterns** are widely used to make code reusable, maintainable, and scalable. Below are the **most popular patterns** you should know as a Drupal/PHP developer.

---

1 Singleton Pattern

Ensures **only one instance** of a class exists.

PHP Example

```
class Database {
  private static $instance = NULL;

  private function __construct() {}

  public static function getInstance() {
    if (!self::$instance) {
      self::$instance = new Database();
    }
    return self::$instance;
  }
}
```

In Drupal

Drupal core services (like database connection) behave similar to singleton via the **service container**.

---

2 Factory Pattern

Creates objects **without exposing the creation logic**.

PHP Example

```
class LoggerFactory {
  public static function create($type) {
    if ($type == 'file') {
      return new FileLogger();
    }
```

```
    return new DatabaseLogger();
  }
}
```

In Drupal

Used heavily in **Plugin Managers**.

Example:

- Block plugins
- Field widgets
- Field formatters

---

3 Dependency Injection (DI)

Objects receive dependencies instead of creating them.

This is a **core pattern in modern Drupal**.

PHP Example

```
class UserService {

  protected $database;

  public function __construct($database) {
    $this->database = $database;
  }

}
```

In Drupal

Using the **service container**:

```
public static function create(ContainerInterface $container) {
  return new static(
    $container->get('entity_type.manager')
  );
}
```

4 Observer Pattern (Event Subscriber)

Objects **listen to events and react when triggered**.

PHP Example

```
interface Observer {
  public function update();
}
```

In Drupal

Used in **Event Subscribers**.

Example:

```
class RouteSubscriber extends RouteSubscriberBase {
  protected function alterRoutes(RouteCollection $collection) {
    // modify routes
  }
}
```

This integrates with **Symfony EventDispatcher**, which Drupal uses internally.

---

5 Strategy Pattern

Allows selecting an **algorithm at runtime**.

PHP Example

```
interface PaymentStrategy {
  public function pay();
}

class PayPalPayment implements PaymentStrategy {
  public function pay() {
    echo "Paid with PayPal";
  }
}
```

In Drupal

Example:

- Different **cache backends**
- Different **queue implementations**

---

6 Plugin Pattern (Very Important in Drupal)

Drupal heavily relies on the **plugin architecture**.

Examples:

- Blocks
- Field formatters
- Field widgets
- Image effects

Example plugin:

```
/**
 * @Block(
 *   id = "custom_block",
 *   admin_label = @Translation("Custom Block")
 * )
 */
class CustomBlock extends BlockBase {
}
```

---

7 Decorator Pattern

Adds functionality **without modifying the original class**.

Example

Drupal services can be decorated.

```
services:
  my.logger:
    decorates: logger.channel.default
```

---

8 Repository Pattern

Used to **separate business logic from data access**.

Example:

```
class UserRepository {
  public function getUserById($id) {
    return \Drupal::entityTypeManager()
      ->getStorage('user')
      ->load($id);
  }
}
```

---

9 MVC Pattern

Common in PHP frameworks.

Drupal uses a modified **MVC pattern** based on **Symfony Framework**.

Structure:

Controller → Service → Entity → Twig Template

🔟 Service Container Pattern

Drupal uses **service containers** to manage dependencies.

Example service:

```
services:
  my_module.user_service:
    class: Drupal\my_module\Service\UserService
```

🧠 Most Important Patterns for Drupal Developers

Pattern	Where Used

Dependency Injection	Services, Controllers
Plugin Pattern	Blocks, Fields
Factory Pattern	Plugin managers
Observer Pattern	Event subscribers
Decorator Pattern	Service overrides

💡 Senior Drupal Interview Tip

If asked:

“Which design patterns does Drupal use?”

Answer:

> Drupal heavily uses **Dependency Injection, Factory, Plugin, Observer, and Decorator patterns**. These are built on top of the **Symfony architecture**.

✅10 real examples from Drupal core where these patterns are implemented (very useful for senior Drupal developer interviews).

Here are **real examples from Drupal core** where important **design patterns** are implemented. These examples are very useful for **Drupal developer interviews and understanding Drupal architecture**. 🚀

1️⃣ Dependency Injection Pattern

Drupal services use **dependency injection via the Symfony container**.

Example from **Drupal Core** controllers.

```
use Symfony\Component\DependencyInjection\ContainerInterface;

class NodeController {

  protected $entityTypeManager;

  public function __construct($entity_type_manager) {
    $this->entityTypeManager = $entity_type_manager;
  }

  public static function create(ContainerInterface $container) {
    return new static(
      $container->get('entity_type.manager')
    );
  }

}
```

📌 Used in:

- Controllers
- Forms
- Event subscribers
- Services

2️⃣ Factory Pattern

Drupal plugin managers create objects dynamically.

Example: **Block plugin manager**

```
$block_manager = \Drupal::service('plugin.manager.block');

$plugin = $block_manager->createInstance('system_powered_by_block');
```

Factory creates the plugin instance.

📌 Used in:

- Block plugins
- Field widgets
- Field formatters
- Image effects

3️⃣ Plugin Pattern (Most Important in Drupal)

Example block plugin:

```
use Drupal\Core\Block\BlockBase;


/**
 * @Block(
 *   id = "custom_block",
 *   admin_label = @Translation("Custom Block")
 * )
 */
class CustomBlock extends BlockBase {


  public function build() {
    return [
      '#markup' => 'Hello Drupal',
    ];
  }
}
```

Plugin system loads it automatically.

📌 Used in:

- Blocks
- Views plugins
- Field types
- Image styles

4️⃣ Observer Pattern (Event Subscriber)

Drupal uses **Symfony EventDispatcher**.

Example event subscriber:

```
use Symfony\Component\EventDispatcher\EventSubscriberInterface;

class KernelSubscriber implements EventSubscriberInterface {

  public static function getSubscribedEvents() {
    return [
      'kernel.request' => 'onRequest',
    ];
  }

  public function onRequest() {
    // Execute logic
  }

}
```

📌 Used for:

- Routing events
- Request events
- Response events

5️⃣ Decorator Pattern

Drupal services can be **decorated (overridden without hacking core)**.

Example service decoration:

```
services:
  my_module.logger:
    decorates: logger.channel.default
    class: Drupal\my_module\LoggerDecorator
```

Now your decorator modifies the original service.

📌 Used in:

- Logging

- Cache
- Authentication

6️⃣ Singleton Pattern

Some Drupal classes ensure a single instance.

Example:

```
\Drupal::database();
```

Internally managed by the **service container**.

📌 Used in:

- Database connection
- Configuration factory

7️⃣ Strategy Pattern

Allows switching behavior dynamically.

Example:

Drupal cache backends.

```
$cache = \Drupal::cache('default');
```

Backend can be:

- Database cache
- Redis cache
- Memcache

Same interface → different implementation.

8️⃣ Repository Pattern

Used in entity storage.

Example:

```
$node = \Drupal::entityTypeManager()
  ->getStorage('node')
  ->load($nid);
```

Entity storage acts like a repository.

---

9 MVC Pattern

Drupal follows Symfony-style MVC.

Example flow:

Route

↓

Controller

↓

Service

↓

Entity

↓

Twig Template

Example controller:

```
class ExampleController {

  public function content() {
    return [
      '#markup' => 'Hello World'
    ];
  }

}
```

Template rendered by **Twig**.

🔟 Service Container Pattern

All major Drupal functionality is exposed as services.

Example service definition:

```
services:
  my_module.user_service:
    class: Drupal\my_module\Service\UserService
```

Access service:

```
$service = \Drupal::service('my_module.user_service');
```

🧠 Most Important Patterns in Drupal Core

Pattern	Drupal Example
Dependency Injection	Controllers & Services
Plugin Pattern	Blocks, Views, Fields
Factory Pattern	Plugin managers
Observer Pattern	Event subscribers
Decorator Pattern	Service overrides
Strategy Pattern	Cache backends

💡 Senior Drupal Interview Answer

If interviewer asks:

“Which design patterns are used in Drupal?”

Answer:

Drupal is built on top of Symfony and heavily uses **Dependency Injection, Plugin, Factory, Observer, Decorator, and Strategy patterns**. These patterns enable extensibility without modifying core code.